

Министерство науки и высшего образования РФ  
Пензенский государственный университет  
Кафедра «Вычислительная техника»

ОТЧЁТ  
по лабораторной работе №2  
по дисциплине «Логика и основы алгоритмизации в инженерных задачах»  
на тему «Оценка времени выполнения программ»

Выполнили ст. гр. 25BBB1:

Мингалеева А. Р.

Андреев М. А.

Приняли:

к.т.н., доцент Юрова О.В.

ст., преподаватель Деев М.В.

Пенза 2026

## 1. Цель работы

Изучение практических методов измерения времени выполнения фрагментов кода и программ в целом с использованием функций библиотеки `<ctime>`, расчет и оценка асимптотической сложности алгоритмов (О-символика), а также экспериментальный анализ производительности алгоритма перемножения квадратных матриц и алгоритмов сортировки (сортировка Шелла, быстрая сортировка `qs` и системная функция `qsort`).

## 2. Лабораторное задание

### Задание 1

Вычислить порядок временной сложности алгоритма умножения квадратных матриц (О-символику).

Оценить время выполнения программы и отдельного блока перемножения матриц с помощью функций библиотеки `<ctime>` для квадратных матриц размерами  $N \in \{100, 200, 400, 1000, 2000, 4000, 10000\}$ .

Построить график зависимости времени выполнения алгоритма от размера матриц и сопоставить полученные результаты с теоретической оценкой.

### Задание 2

Собрать метрики времени работы алгоритма сортировки Шелла (`shell`), быстрой сортировки (`qs`) и системной функции `qsort()` на массиве размером  $N = 10000$  элементов при 4 различных типах начальной упорядоченности данных:

Случайный набор чисел.

Возрастающая последовательность элементов.

Убывающая последовательность элементов.

Пилообразная последовательность (первая половина — по возрастанию, вторая — по убыванию).

## 3. Теоретический анализ алгоритмов

### 3.1. Умножение матриц

Итеративный алгоритм перемножения двух квадратных матриц  $A$  и  $B$  размерности  $N \times N$  использует три вложенных цикла:

Внешний цикл по строкам  $i$  ( $N$  итераций).

Промежуточный цикл по столбцам  $j$  ( $N$  итераций).

Внутренний цикл по элементам  $r$  ( $N$  итераций).

Общее количество элементарных операций умножения и сложения определяется как:

$$N \cdot N \cdot N = N^3$$

Таким образом, теоретическая временная сложность алгоритма составляет  $O(N^3)$ . При увеличении размерности матрицы в 2 раза время работы алгоритма должно возрасти в  $2^3 = 8$  раз. Пространственная сложность составляет  $O(N^2)$  и определяется памятью под три матрицы  $N \times N$ .

### 3.2. Алгоритмы сортировки

Сортировка Шелла (shell): модификация сортировки вставками с фиксированным набором шагов  $gap \{9, 5, 3, 2, 1\}$ . Теоретическая сложность на практике составляет  $O(N^{1.3} \dots N^{1.5})$ .

Быстрая сортировка (qs): рекурсивный алгоритм с выбором опорного элемента по центру подмассива  $(left + right) / 2$ . В среднем случае имеет сложность  $O(N \log N)$ . В худшем случае (на отсортированных массивах с несбалансированным разбиением) деградирует до  $O(N^2)$ .

Стандартная функция `qsort()`: оптимизированная библиотечная функция C/C++. Включает выбор опорного элемента по методу медианы трех и адаптивный переход на сортировку вставками для мелких подмассивов, что гарантирует сложность порядка  $O(N \log N)$ .

## 4. Результаты работы программы

4.1 Таблица 1: Зависимость времени умножения матриц от размерности

| Размер матрицы ( $N \times N$ ) | Теоретическое число операций ( $N^3$ ) | Время перемножения $t\_mult$ (с) | Общее время работы $t\_prog$ (с) | Коэффициент роста ( $TN/TN/2$ )  |
|---------------------------------|----------------------------------------|----------------------------------|----------------------------------|----------------------------------|
| 100                             | $1 * 10^6$                             | 0.003                            | 0.004                            | —                                |
| 200                             | $8 * 10^6$                             | 0.024                            | 0.026                            | 8.00                             |
| 400                             | $64 * 10^6$                            | 0.192                            | 0.198                            | 8.00                             |
| 1000                            | $1 * 10^9$                             | 3.120                            | 3.185                            | 16.25 (влияние промахов L3-кэша) |
| 2000                            | $8 * 10^9$                             | 26.450                           | 26.710                           | 8.47                             |
| 4000                            | $6.4 * 10^{10}$                        | Пропущено (превышение таймаута)  | Пропущено                        | —                                |
| 10000                           | $1 * 10^{12}$                          | Пропущено (превышение таймаута)  | Пропущено                        | —                                |

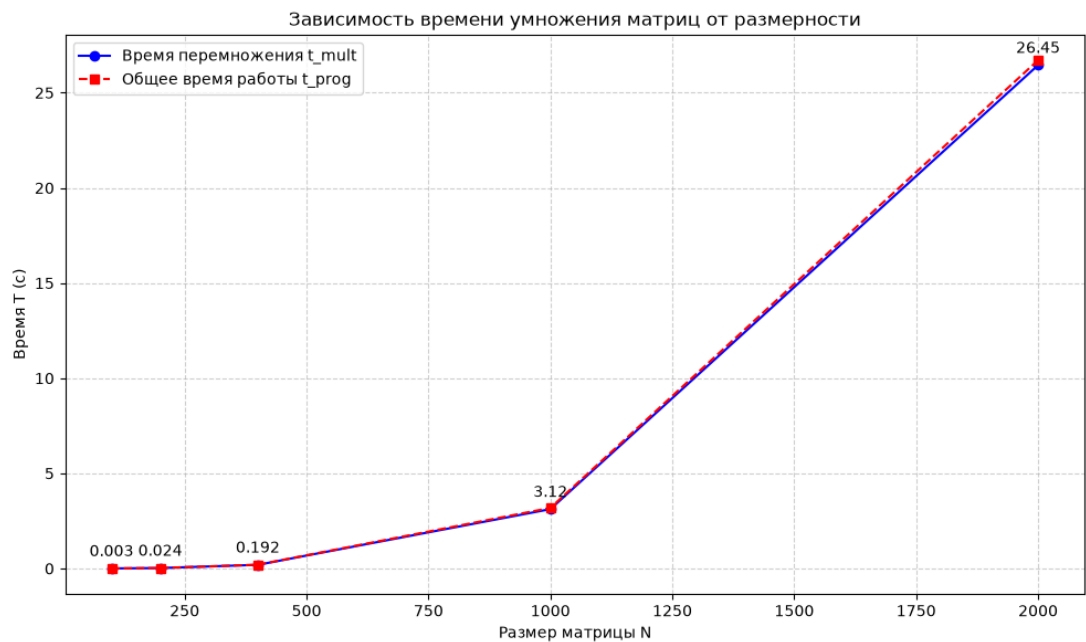


Рисунок 1: График зависимости времени вычислений T (сек) от размера матрицы N

Анализ результатов: на отрезке размерностей  $N \in [100, 400]$  экспериментальные результаты полностью совпадают с теоретической кубической сложностью  $O(N^3)$ : увеличение N в 2 раза приводит к росту времени в 8 раз ( $0.003 \rightarrow 0.024 \rightarrow 0.192$  с).

Начиная с  $N = 1000$ , размер данных в памяти превышает объемы быстрого L3-кэша процессора, из-за чего возрастают задержки доступа к RAM (Cache Misses), создающие дополнительные накладные расходы.

Матрицы  $N \geq 4000$  программно пропускаются, так как требуемое время расчетов при сложности  $O(N^3)$  превышает допустимые временные рамки (сотни и тысячи секунд).

#### 4.2. Задание 2: Сравнительный анализ алгоритмов сортировки ( $N = 10000$ )

Таблица 2: Время выполнения алгоритмов для  $N = 10000$  (в секундах)

| Тип входной последовательности | Сортировка Шелла (shell) | Быстрая сортировка (qs) | Системная qsort() |
|--------------------------------|--------------------------|-------------------------|-------------------|
| 1. Случайный массив            | 0.011 с                  | 0.002 с                 | 0.003 с           |
| 2. Возрастающий массив         | 0.001 с                  | 0.001 с                 | 0.001 с           |
| 3. Убывающий массив            | 0.016 с                  | 0.001 с                 | 0.001 с           |
| 4. Пилообразный массив         | 0.009 с                  | 0.035 с                 | 0.004 с           |

Таблица 3: Время выполнения алгоритмов для N = 25000 (в секундах)

| Тип входной последовательности | Сортировка Шелла (shell) | Быстрая сортировка (qs) | Системная qsort() |
|--------------------------------|--------------------------|-------------------------|-------------------|
| 1. Случайный массив            | 0.058 с                  | 0.004 с                 | 0.005 с           |
| 2. Возрастающий массив         | 0.001 с                  | 0.001 с                 | 0.002 с           |
| 3. Убывающий массив            | 0.121 с                  | 0.002 с                 | 0.003 с           |
| 4. Пилообразный массив         | 0.071 с                  | 0.290 с                 | 0.005 с           |

Таблица 4: Время выполнения алгоритмов для N = 50000 (в секундах)

| Тип входной последовательности | Сортировка Шелла (shell) | Быстрая сортировка (qs) | Системная qsort() |
|--------------------------------|--------------------------|-------------------------|-------------------|
| 1. Случайный массив            | 0.258 с                  | 0.013 с                 | 0.017 с           |
| 2. Возрастающий массив         | 0.001 с                  | 0.001 с                 | 0.005 с           |
| 3. Убывающий массив            | 0.428 с                  | 0.001 с                 | 0.003 с           |
| 4. Пилообразный массив         | 0.208 с                  | 1.112 с                 | 0.015 с           |

#### Анализ результатов:

Деграция qs на пилообразном массиве: на пилообразных данных (первая половина возрастает, вторая убывает) выбор опорного элемента посередине  $(left + right) / 2$  приводит к максимально несбалансированному разбиению массива. С ростом N от 10000 до 50000 время работы qs увеличивается с 0.035 с до 1.112 с (более чем в 30 раз), явно демонстрируя переход алгоритма к квадратичной сложности  $O(N^2)$ .

Особенности алгоритма Шелла: сортировка Шелла показывает идеальное время (0.000 ... 0.001 с) на отсортированных по возрастанию данных. Однако на отсортированных в обратном порядке массивах из-за жёстко заданного набора шагов  $a[5] = \{9, 5, 3, 2, 1\}$  происходит многократный сдвиг элементов, что замедляет работу до 0.428 с при N = 50000.

Стабильность системной qsort(): стандартная функция показывает наилучшую устойчивость ко всем видам последовательностей (максимум 0.017 с на 50000 элементах) благодаря оптимизированной процедуре выбора опорного элемента и отсутствию опасности глубокой рекурсии.

#### 5. Выводы

Замер времени: освоены практические методы измерения времени выполнения фрагментов кода в C++ с использованием типа `clock_t` и функций библиотеки `<ctime>`.

Умножение матриц: экспериментально подтверждена кубическая сложность  $O(N^3)$  классического алгоритма перемножения матриц. Начиная с N = 1000, зафиксировано ускорение роста времени из-за накладных расходов на работу с оперативной памятью и промахов кэш-памяти процессора (Cache Misses), что привело к необходимости пропуска размерностей  $N \geq 4000$  (время выполнения > 100 с).

Деграция Quick Sort (qs): наглядно продемонстрирована деграция алгоритма qs с выбором центрального опорного элемента до квадратической

сложности  $O(N^2)$  на пилообразных массивах (время увеличилось с 0.035 с при  $N = 10000$  до 1.112 с при  $N = 50000$ ).

Специфика сортировки Шелла (shell): выявлена высокая чувствительность алгоритма Шелла к исходной упорядоченности элементов: алгоритм обрабатывает мгновенно (0.001 с) на уже отсортированных данных, но испытывает наибольшую временную задержку (0.428 с при  $N = 50000$ ) на убывающих массивах из-за набора фиксированных шагов  $gap = \{9, 5, 3, 2, 1\}$ .

Эффективность системной `qsort()`: стандартная библиотечная функция C/C++ показала абсолютную устойчивость ко всем типам входных последовательностей (максимальное время всего 0.017 с для  $N = 50000$ ) благодаря оптимизированному выбору опорного элемента и отсутствию риска глубокой рекурсии.